CSA1735 - LAB PROGRAMS

6. Vacuum Cleaner Problem

Aim: To write a Python program to solve the Vacuum Cleaner problem using a simple reflex agent.

Algorithm:

1. Represent two rooms, A and B, as clean or dirty.
2. Start the vacuum cleaner in room A.
3. If the current room is dirty, clean it.
4. Move to the other room.
5. Stop when both rooms are clean.

Program:

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}

location = 'A'


while 'Dirty' in rooms.values():

    print("Vacuum at room", location)


    if rooms[location] == 'Dirty':

        rooms[location] = 'Clean'

        print("Room", location, "cleaned")


    if location == 'A':

        location = 'B'

    else:

        location = 'A'
```

print("All rooms are clean")

print(rooms)

output:

```
= RESTART: C:/Users/admin/Documents/AI/Exp-6.py
Vacuum at room A
Room A cleaned
Vacuum at room B
Room B cleaned
All rooms are clean
{'A': 'Clean', 'B': 'Clean'}
```

7. Breadth-First Search (BFS)

Aim: To write a Python program to implement Breadth-First Search.

Algorithm:

1. Create a graph using an adjacency list.
2. Initialize a queue with the starting node.
3. Mark the starting node as visited.
4. Remove a node from the front of the queue and print it.
5. Add all its unvisited neighboring nodes to the queue.
6. Repeat until the queue is empty.

Program:

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])

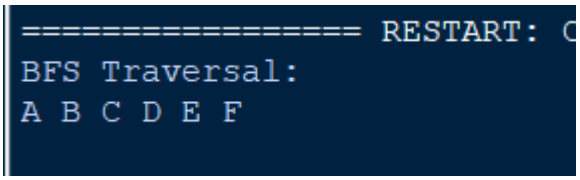
    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

print("BFS Traversal:")
bfs(graph, 'A')
```

Output:



BFS Traversal:
A B C D E F

8. Depth-First Search (DFS)

Aim: To write a Python program to implement Depth-First Search.

Algorithm:

1. Represent the graph using an adjacency list.
2. Start from the given node.
3. Mark the node as visited and print it.
4. Visit each unvisited neighboring node recursively.
5. Continue until all reachable nodes are visited.

Program:

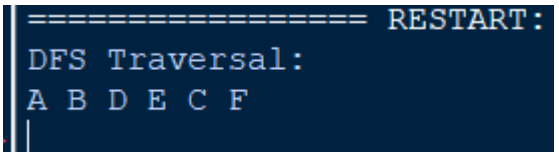
```
def dfs(graph, node, visited):
    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

print("DFS Traversal:")
dfs(graph, 'A', set())
```

Output:



DFS Traversal:
A B D E C F

9. Travelling Salesman Problem (TSP)

Aim: To write a Python program to solve the Travelling Salesman Problem using brute force.

Algorithm:

1. Represent the cities and distances using a cost matrix.
2. Fix the starting city as city 0.
3. Generate all possible permutations of the remaining cities.
4. Calculate the cost of each complete tour, including the return to the starting city.
5. Find and print the tour with minimum cost.

Program:

```
from itertools import permutations

def tsp(graph):
    n = len(graph)
    cities = range(1, n)
    min_cost = float('inf')
    best_path = []

    for p in permutations(cities):
```

```
        path = (0,) + p + (0,)
        cost = sum(
            graph[path[i]][path[i+1]]
            for i in range(n)
        )

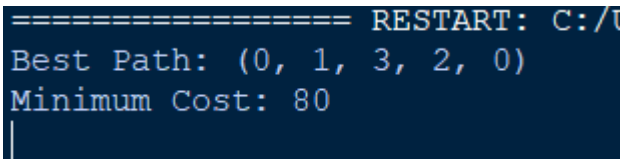
        if cost < min_cost:
            min_cost = cost
            best_path = path

    print("Best Path:", best_path)
    print("Minimum Cost:", min_cost)

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

tsp(graph)
```

Output:



Best Path: (0, 1, 3, 2, 0)
Minimum Cost: 80

10. A* Search Algorithm

Aim: To write a Python program to implement the A* search algorithm.

Algorithm:

1. Represent the graph with edge costs and define a heuristic for each node.
2. Initialize the start node with cost $g = 0$.
3. Calculate the evaluation function $f(n) = g(n) + h(n)$.
4. Select the node with the lowest $f(n)$.
5. Update the costs of its neighboring nodes.
6. Repeat until the goal node is reached.

Program:

```
import heapq

def astar(graph, heuristic, start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

        if node in visited:
            continue

        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(
                    pq, (new_f, new_g, neighbor,
                         path + [neighbor])
                )

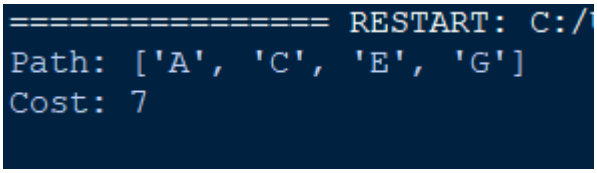
    print("No path found")

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1)],
    'D': [('G', 5)],
    'E': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 5, 'B': 4, 'C': 2,
    'D': 4, 'E': 1, 'G': 0
}

astar(graph, heuristic, 'A', 'G')
```

Output:



Path: ['A', 'C', 'E', 'G']
Cost: 7